

[Algorithm]

스택

인하대학교 소프트웨어융합공학과

김종훈 교수



이 장에서 다룰 내용



스택



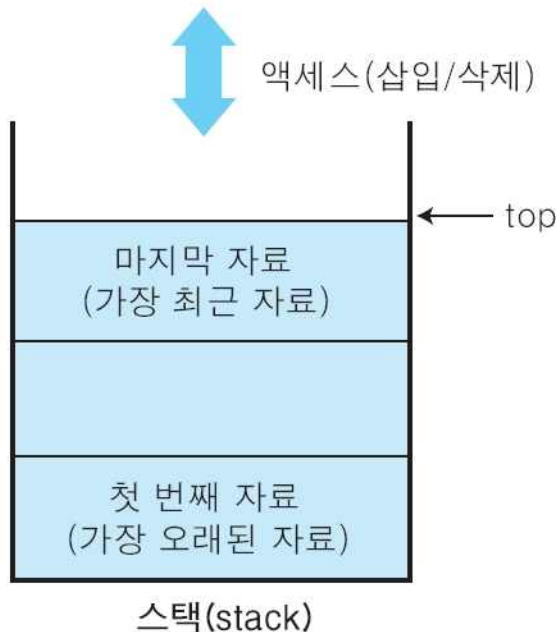
스택의 추상 자료형



스택의 구현

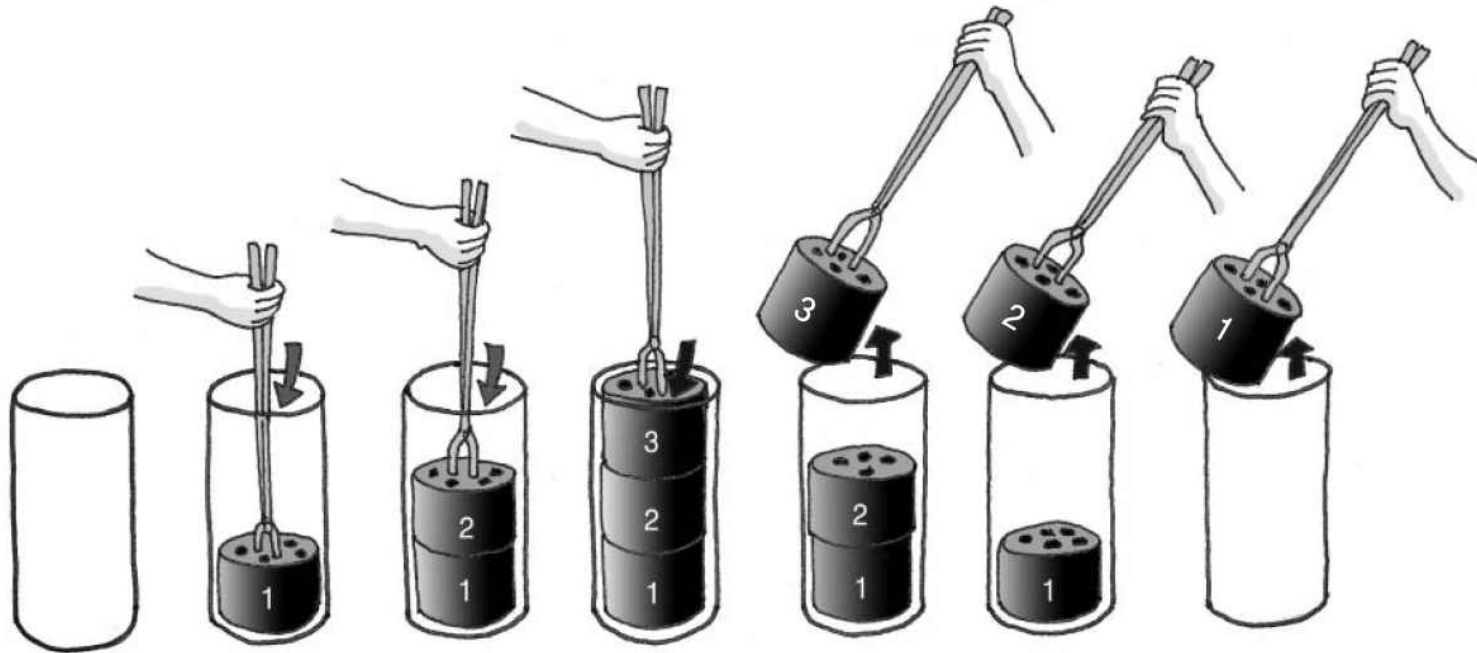
❖ 스택(stack)

- 접시를 쌓듯이 자료를 차곡차곡 쌓아 올린 형태의 자료구조
- 스택에 저장된 원소는 top으로 정한 곳에서만 접근 가능
 - top의 위치에서만 원소를 삽입하므로 먼저 삽입한 원소는 밑에 쌓이고, 나중에 삽입한 원소는 위에 쌓인다.
 - 마지막에 삽입(Last-In)한 원소는 맨 위에 쌓여 있다가 가장 먼저 삭제(First-Out)된다. ➡ **후입선출 구조** (LIFO, Last-In-First-Out)



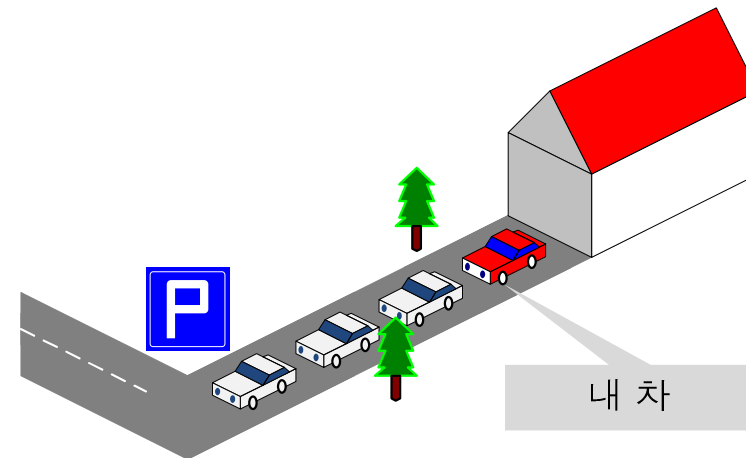
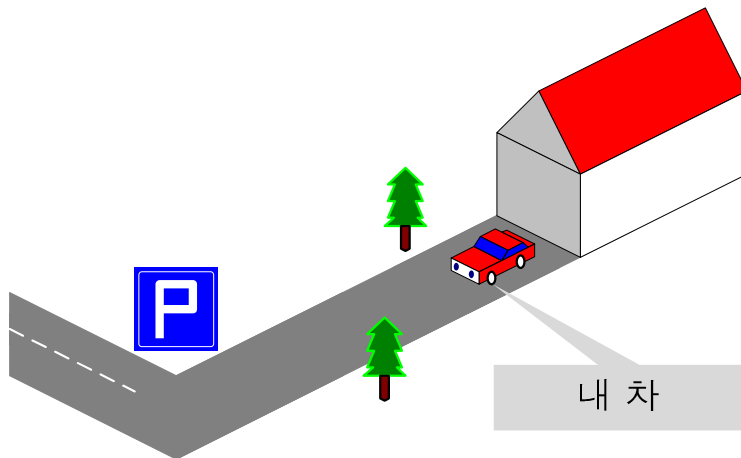
■ 후입선출 구조의 예1 : 연탄 아궁이

- 연탄을 하나씩 쌓으면서 아궁이에 넣으므로 마지막에 넣은 3번 연탄이 가장 위에 쌓여 있다.
- 연탄을 아궁이에서 꺼낼 때에는 위에서부터 하나씩 꺼내야 하므로 마지막에 넣은 3번 연탄을 가장 먼저 꺼내게 된다.



■ 후입선출 구조의 예2 :스택 주차장

- 가장 안쪽에 주차된 내 차를 빼려면?
- 내 차보다 나중에 주차된 차를 모두 빼야 한다.



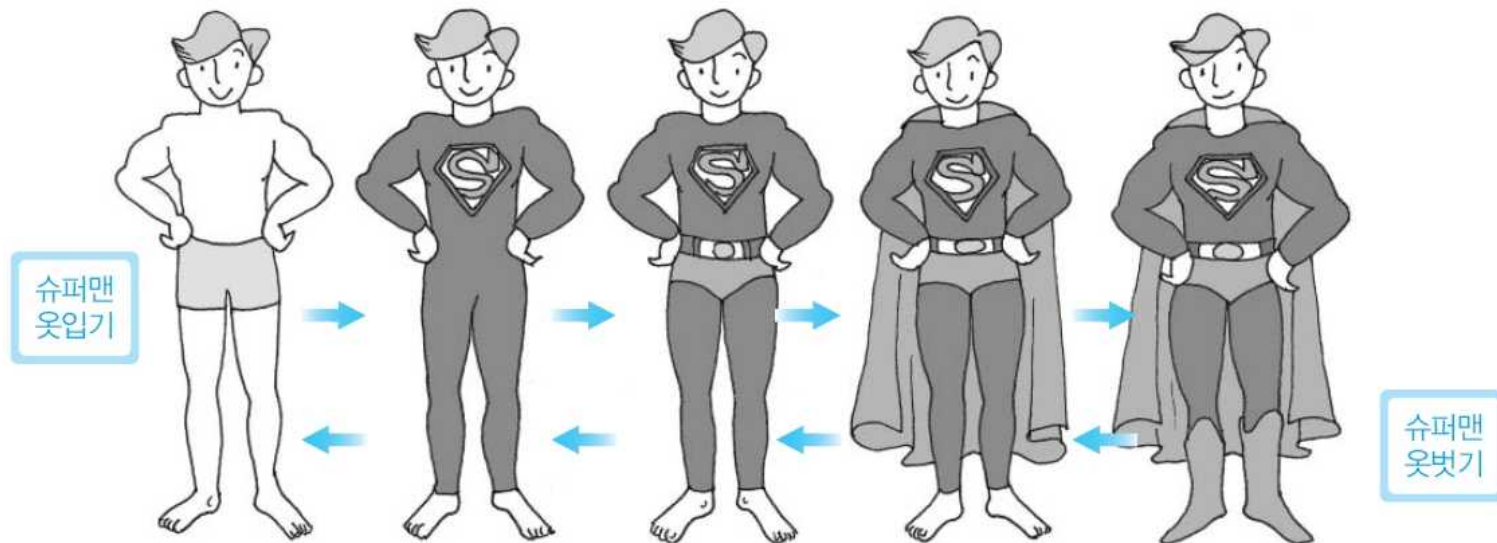
■ 후입선출 구조의 예3 : 슈퍼맨의 옷 갈아입기

- 슈퍼맨이 옷을 벗는 순서

① 장화 → ②망토 → ③빨간팬츠 → ④파란옷

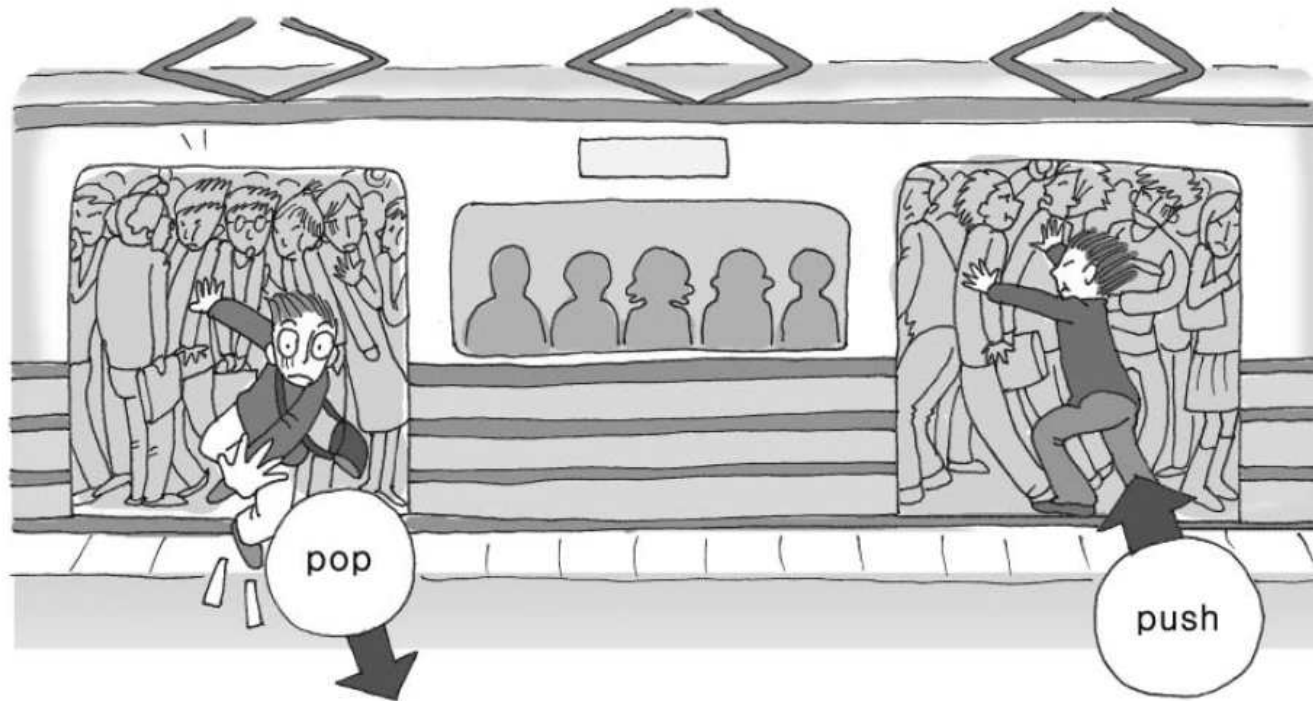
- 슈퍼맨이 옷을 입는 순서

④ 파란옷 → ③빨간팬츠 → ②망토 → ①장화



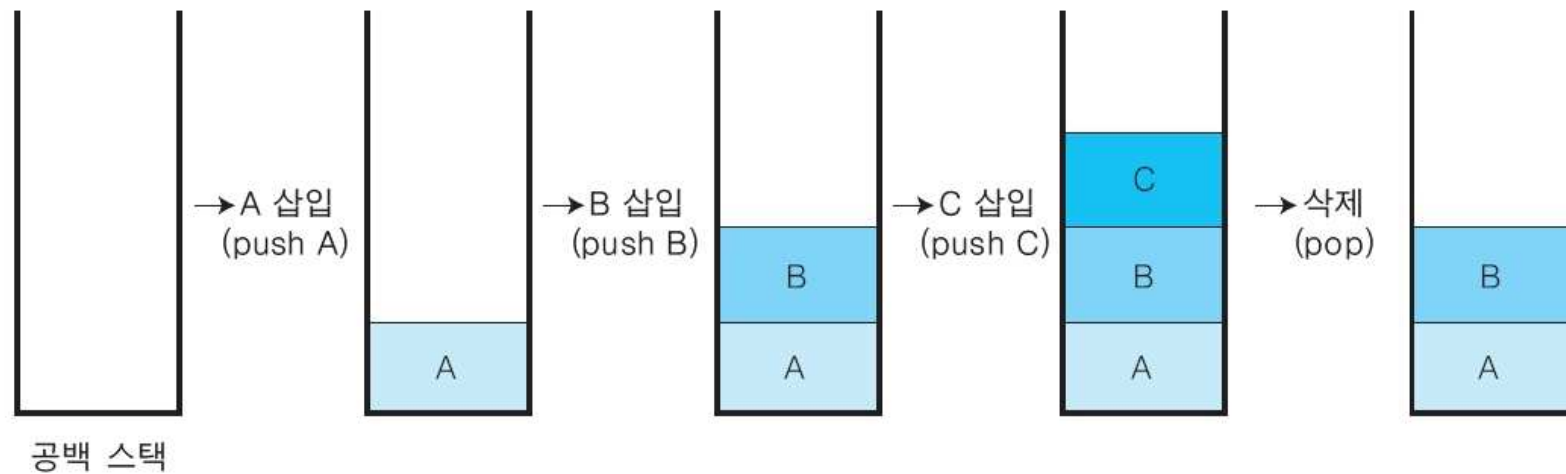
❖ 스택의 연산

- 스택에서의 삽입 연산 : **push**
- 스택에서의 삭제 연산 : **pop**



■ 스택에서의 원소 삽입/삭제 과정

- 공백 스택에 원소 A, B, C를 순서대로 삽입하고 한번 삭제하는 동안의 스택 변화



□ 스택의 추상 자료형

■ 스택에 대한 추상 자료형

ADT Stack

데이터 : 0개 이상의 원소를 가진 유한 순서 리스트

연산 : $S \in \text{Stack}; \text{item} \in \text{Element};$

createStack() ::= create an empty Stack;

// 공백 스택을 생성하는 연산

isEmpty(S) ::= if (S is empty) then return true
else return false;

// 스택 S가 공백인지 아닌지를 확인하는 연산

push(S, item) ::= insert item onto the top of S;

// 스택 S의 top에 item(원소)을 삽입하는 연산

pop(S) ::= if (isEmpty(S)) then return error
else { delete and return the top item of S };

// 스택 S의 top에 있는 item(원소)을 스택에서 삭제하고 반환하는 연산

delete(S) ::= if (isEmpty(S)) then return error
else delete the top item;

// 스택 S의 top에 있는 item(원소)을 삭제하는 연산

peek(S) ::= if (isEmpty(S)) then return error
else return (the top item of the S);

// 스택 S의 top에 있는 item(원소)을 반환하는 연산

End Stack

❖ 스택의 push 알고리즘

① $top \leftarrow top+1;$

- 스택 S에서 top이 마지막 자료를 가리키고 있으므로 그 위에 자료를 삽입하려면 먼저 top의 위치를 하나 증가
- 만약 이때 top의 위치가 스택의 크기(stack_SIZE)보다 크다면 오버플로우(overflow)상태가 되므로 삽입 연산을 수행하지 못하고 연산 종료

② $S(top) \leftarrow x;$

- 오버플로우 상태가 아니라면 스택의 top이 가리키는 위치에 x 삽입

```
push(S, x)
  top  $\leftarrow$  top+1; // ①
  if (top > stack_SIZE) then overflow;
    else
      S(top)  $\leftarrow$  x; // ②
end push()
```

❖ 스택의 pop 알고리즘

① **return S(top);**

- 스택이 공백 스택이 아니라면 top이 가리키는 원소를 먼저 반환

② **top ← top-1;**

- 스택의 top 원소를 반환하였으므로 top의 위치는 그 아래의 원소로 변경하기 위해 top의 위치를 하나 감소

```
pop(S)
  if (top = 0) then error;
  else {
    return S(top); // ①
    top ← top-1; // ②
  }
end pop()
```

❖ 순차 자료구조를 이용한 스택의 구현

■ 순차 자료구조인 1차원 배열을 이용하여 구현

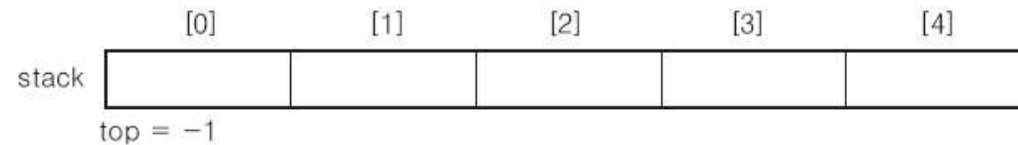
- 스택의 크기 : 배열의 크기
- 스택에 저장된 원소의 순서 : 배열 원소의 인덱스
 - 인덱스 0번 : 스택의 첫번째 원소
 - 인덱스 $n-1$ 번 : 스택의 n 번째 원소



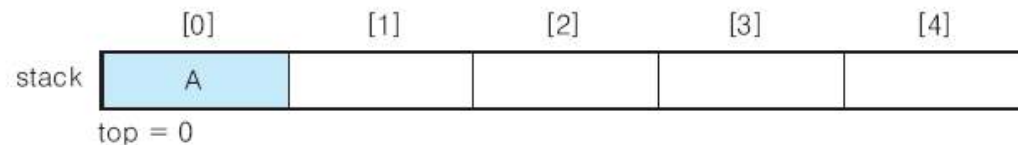
- 변수 `top` : 스택에 저장된 마지막 원소에 대한 인덱스 저장
 - 공백 상태 : `top = -1` (초기값)
 - 포화 상태 : `top = n-1`

크기가 5인 1차원 배열의 스택에서의 연산 수행과정

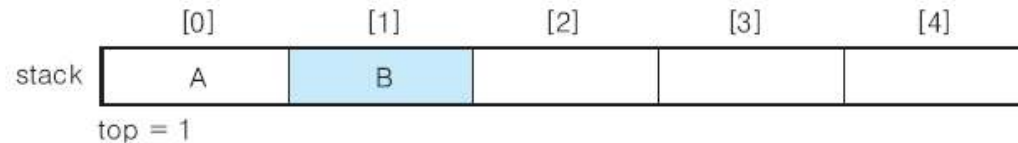
① 공백 스택 생성 : `create(stack, 5);`



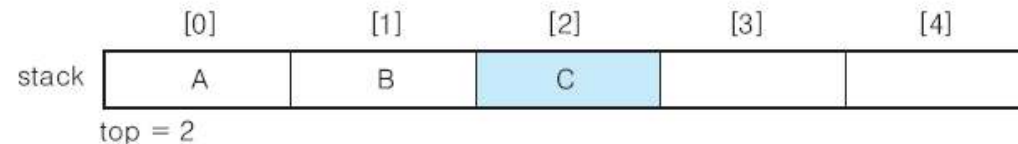
② 원소 A 삽입 : `push(stack, A);`



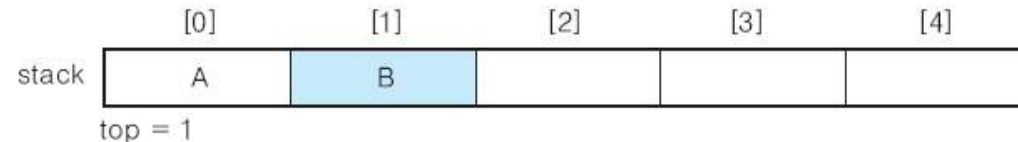
③ 원소 B 삽입 : `push(stack, B);`



④ 원소 C 삽입 : `push(stack, C);`



⑤ 원소 삭제 : `pop(stack);`



□ 스택의 구현

- 순차 자료구조로 구현한 스택의 장점

- 순차 자료구조인 1차원 배열을 사용하여 쉽게 구현

- 순차 자료구조로 구현한 스택의 단점

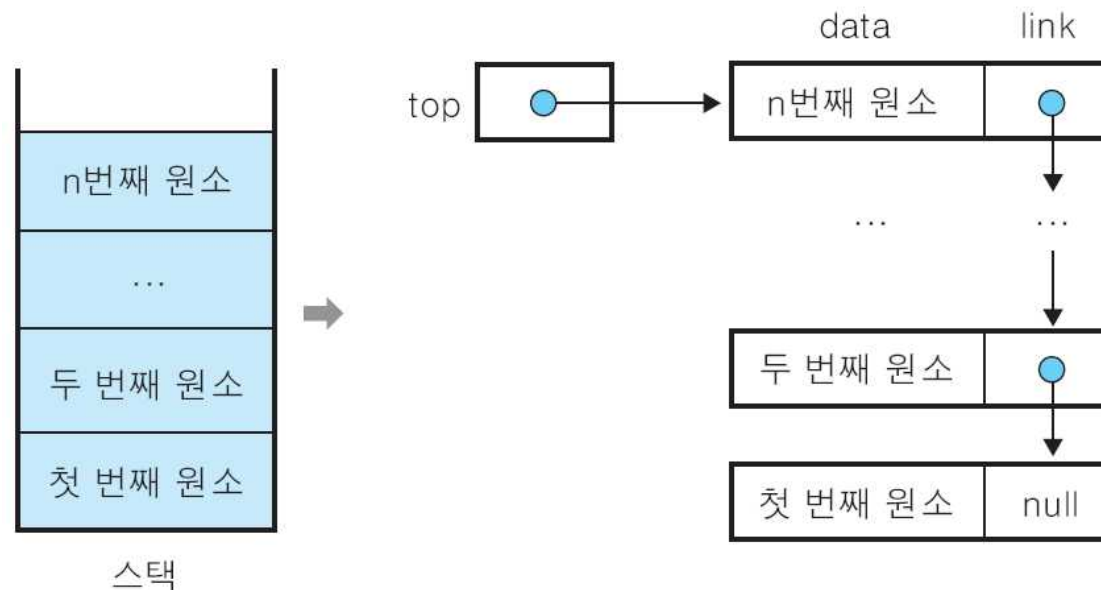
- 물리적으로 크기가 고정된 배열을 사용하므로 스택의 크기 변경 어려움
- 순차 자료구조의 단점을 그대로 가지고 있다.

□ 스택의 구현

❖ 연결 자료구조를 이용한 스택의 구현

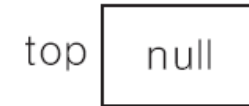
■ 단순 연결 리스트를 이용하여 구현

- 스택의 원소 : 단순 연결 리스트의 노드
 - 스택 원소의 순서 : 노드의 링크 포인터로 연결
 - push : 리스트의 마지막에 자료 삽입 (왼쪽그림)
 - Pop : 리스트의 마지막 자료 삭제 (왼쪽그림)
- 변수 top : 단순 연결 리스트의 마지막 노드를 가리키는 포인터 변수
 - 초기 상태 : $top = null$

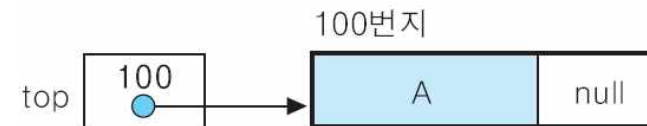


■ 단순 연결 리스트의 스택에서의 연산 수행과정

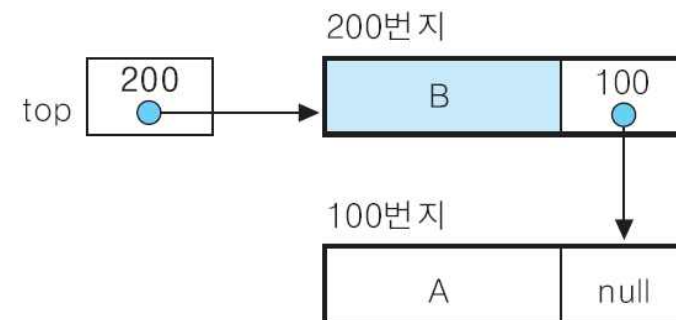
① 공백 스택 생성 : `create(stack);`



② 원소 A 삽입 : `push(stack, A);`

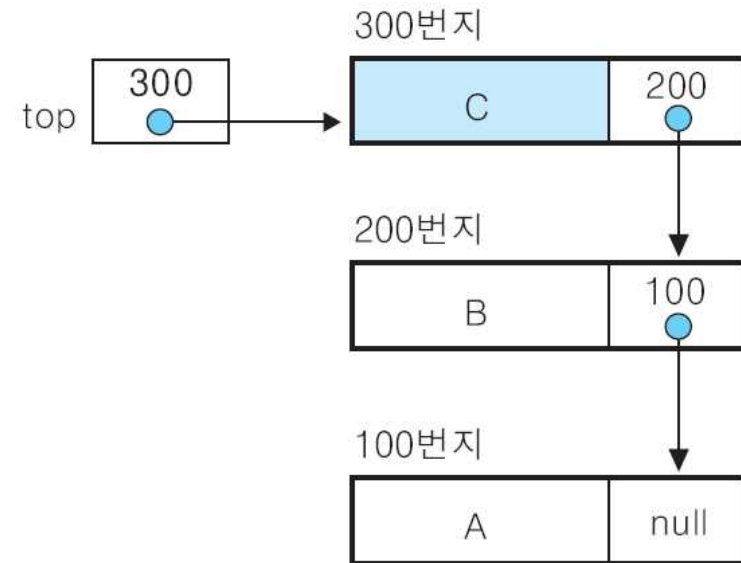


③ 원소 B 삽입 : `push(stack, B);`

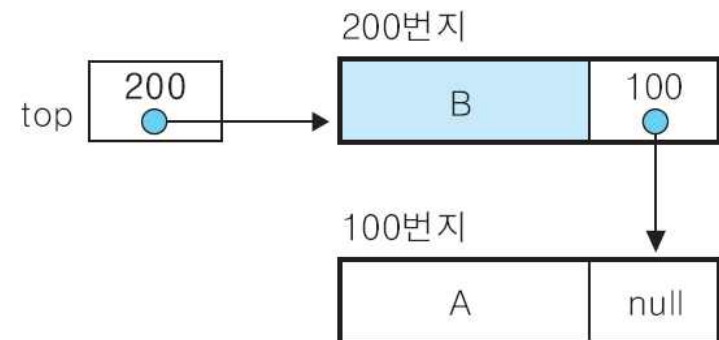


□ 스택의 구현

④ 원소 C 삽입 : `push(stack, C);`



⑤ 원소 삭제 : `pop(stack);`





Thank You